Luisito L. Agustin

20260319

Project 3
Linear Time-Invariant System Simulator

Project 3
Linear Time-Invariant System Simulator

Due: start of class time, April 30, 2026

Progress Report 1	Mar. 26
Progress Report 2	Apr. 16
Progress Report 3	Apr. 23
Early Work Submission	Apr. 27

DSP Project

Linear Time-Invariant System Simulator

Luisito L. Agustin
2023 10 15

General Specifications

Develop an application in C++ for simulating linear time-invariant (LTI) systems.

Detailed Specifications

A general form of the input-output relationship for a linear time-invariant (LTI) system is given by the linear constant-coefficient difference equation

$$y(n) = -\sum_{k=1}^{N} a_k\, y(n-k) + \sum_{k=0}^{M} b_k\, x(n-k) \ ,$$

where the $\{a_k\}$ and $\{b_k\}$ are constant parameters that specify the system and are independent of the input $x(n)$ and output $y(n)$. For a fixed value n, the equation specifies how the output $y(n)$ is computed based on the parameters $\{ a_k \}$ and $\{ b_k \}$, current and past inputs $\{ x(n-k) \}$ and past outputs $\{ y(n-k) \}$.

The parameters of an LTI system for simulation will be read from an LTI System file.

The application starts by showing information about itself including information about how to get “help”. The application opens a log file named “ltisim-log.txt” in append mode. The application will be appending entries to the file. The file is created if it does not exist. If there is a problem with opening or creating the file, the application will continue working but will not log anything; a message is displayed informing the user of the error.

The application then implements a program loop, where each iteration of the loop consists of the following actions, in sequence:

* display a prompt
* obtain input from the user in the form of a string
* act on user input (possibly exit)
* show results or provide feedback (unless application exited)

The possible inputs from the user and the actions to be taken are as follows:

input	action
help (The word "help" typed at the prompt provided by the application)	Provide information on how to use the application.
system *filename* (The word "system" typed at the prompt provided by the application; The rest of the line must contain a valid filename.)	Extract coefficients of an LTI system from *filename*. If successful, the LTI system specified will be the system to be simulated; it replaces the current LTI system being simulated. (Replace the current LTI system only if a new one has been validated.) The application adds the line "new system" to the log file followed by the required components of the LTI System File (one entry per line as in the actual LTI System File, without any comments), followed by the line "ready". Clear all initial conditions to 0.0 . Provide information on screen on whether an LTI system was successfully obtained from the file.
any floating point number (A floating point number is typed at the prompt provided by the application.)	The number is treated as the next input to the system. The next output is computed. Print (only) the output on the screen (The input typed would still be visible.) Log the input-output pair. Use " \t" (a space followed by a tab) as separator between the input and output values. Print an error message if no LTI system has been defined yet.
signal *filename* (The word "signal" typed at the prompt provided by the application; The rest of the line must contain a valid filename.)	Try to extract a signal from *filename* if it is a valid signal file. If a signal is successfully extracted, the signal serves as input to the LTI system, one sample at a time. The starting index is ignored. Log each input-output pair. Use " \t" (a space followed by a tab) as separator between the input and output value. For signal files with duration less than 10, print each input-output pair onscreen. For longer signals, provide only a summary of the number of inputs simulated.
clear (The word "clear" typed at the prompt provided by the application)	Clear the application's memory of previous inputs and outputs to 0 Log the line "clear". Simulation restarts. Do NOT clear the screen.

exit (The word “exit” typed at the prompt provided by the application)	The application exits.
cls (The word “cls” typed at the prompt provided by the application)	clears the screen; no other output is produced. Implementing this command is optional. This command will only affect what is visible on the screen. It should not have any effect whatsoever on the simulation in progress.

The commands listed must be supported as specified. No alternate commands are allowed. No menu system will be provided. The user is expected to type the commands as specified.

Do not modify any filename provided by the user. Do not assume, modify, require, nor add any extension. Use whatever filename is provided.

All actions are initiated by the user, never by the system. For instance, the system should not ask the user for a file specifying an LTI system at the start even though that may be the most logical way to start using the simulator. The system may only show the user what options are available. Specifying input signals for a system or clearing the state of the simulator may only be done if a system has been loaded already. Display error messages if the user initiates these actions without having specified an LTI system yet.

The starting index obtained when reading a signal file for inputs to a system shall be ignored in the simulation. Signal values read from a file will be treated as the next inputs to the LTI system being simulated regardless of the starting index in the file.

The application never terminates, nor goes into an infinite loop, nor crashes, nor hangs on any input, whether valid or invalid, except that the user may terminate the application intentionally.

Never clear the screen. Previous interactions between the application and the user must remain visible, except that the screen should be cleared if the cls command is availed of.

Clear the application’s memory of previous inputs and outputs to 0 when a new system is loaded.

Relevant portions of signal files should be validated completely. Signal values extracted from signal files should be placed in dynamically allocated arrays.

Write a C++ function, with appropriate parameters, for processing signal files. Any C++ vectors used in processing signal files must be local variables of this function; these vectors will go out of scope when this function exits. Relevant data in these vectors must be copied to separate dynamically allocated arrays before the function exits.

Relevant portions of LTI system files should be validated completely. LTI system coefficients extracted from LTI system files should be placed in dynamically allocated arrays.

Write a C++ function, with appropriate parameters, for processing LTI system files. Any C++ vectors used in processing LTI system files must be local variables of this function; these vectors will go out of scope when this function exits. Relevant data in these vectors must be copied to separate dynamically allocated arrays before the function exits.

For the simulation, write a single C++ function that accepts among its parameters a single input sample value to the system and the state of the LTI system represented by the required dynamically allocated fixed-size arrays for storing the coefficients, inputs, and outputs of the LTI system. The function computes the resulting output and writes this to a parameter or returns this value, or both. A similar function, that accepts an array of signal values instead of a single value, is optional.

Use dynamically allocated fixed-size arrays for storing the coefficients of an LTI system and its previous inputs and outputs. The allocation should be done after the numbers of recursive and nonrecursive coefficients have been determined. If the simulator already has coefficients from an earlier LTI system, the dynamically allocated array is deleted only if a newly specified LTI system has been validated.

For the simulation to be realistic and consistent with how LTI systems might operate in hardware, do not use C++ vectors for storing the inputs to and outputs of the LTI system being simulated. The arrays that store these inputs and outputs do not grow like C++ vectors in an actual implementation.

References

[Proakis] (book) John Proakis and Dimitris Manolakis: Digital Signal Processing, 4th ed, 2007.

Revision History - Linear Time-Invariant System Simulator

2020 12 02	first version
2021 11 18	clearer instructions, required dynamically allocated fixed-size arrays
2022 04 15	revised commands
2023 03 22	removed submission instructions from detailed specifications
2023 10 15	explicit prohibition on use of C++ vectors

Variable Specifications

Sections marked "(variable)" and specifications and instructions from this point onwards may vary from one use of the document to the next. These are not tracked as revisions to the project specifications.

Group Work

Work on the C++ code may be done by groups. Documentation and testing must be done individually.

It is expected that C++ code submitted by groupmates may be similar or exactly the same. Points earned for the entire project will be subject to penalties for unauthorized collaboration if documentation submitted by at least two students contain at least one distinctive phrase, image, or item in common.

Basic Test

LTI system file for basic test (lab5.system)

```
3
2
0.3290316064918401
-0.31460820551056307
0.11124319871419965
-1.439788397628722
0.5654549973241987
```

with comments

```
3                          nNonRecursiveCoefs = M+1 = 3; M = 2
2                          nRecursiveCoefs = N = 2
0.3290316064918401 b0 = 0.3290316064918401
-0.31460820551056307       b1 = -0.31460820551056307
0.11124319871419965        b2 = 0.11124319871419965
-1.439788397628722         a1 = -1.439788397628722
0.5654549973241987 a2 = 0.5654549973241987
from DSP Lab Activity: Magnitude Response of a Biquad Filter
```

Signal File for basic test (test.txt)

```
0.2902846772544620
0.3826834323650900
0.4713967368259980
```

The LTI system is first loaded. The simulator is then given the following inputs:

0.0000000000000000
0.0980171403295606
0.1950903220161280
0.2902846772544620
0.3826834323650900
0.4713967368259980

The first 3 inputs are typed interactively one at a time at the application prompt. The next 3 are provided to the simulator thru a signal file for the basic test.

A screenshot similar to the following should then be obtained:

```
C:\temp>ltisim
LTISim
Type "help" for more information.
ltisim> 0
Cannot simulate. No system defined.
ltisim> system lab5.system
System obtained from lab5.system. recursive coefs: 2, nonrecursive coefs: 3
ltisim> 0
         0
ltisim> 0.0980171403295606
         0.0322507
ltisim> 0.1950903220161280
         0.0797881
ltisim> signal test.txt
Signal obtained from test.txt. start index: 0, duration: 3
0.290285         0.141681
0.382683         0.215166
0.471397         0.29668
ltisim>
```

Submission

Do the basic test exactly as specified with exactly the same data provided. Obtain a screenshot of the application on the command prompt showing the output generated from doing the basic test.

Implementation and Testing Environment Report

Provide details of at least one platform used in implementing and testing the project.

* name and version of compiler
* name and version of integrated development environment (IDE)
* operating system name/distribution and version

screenshot of compilation

The screenshot must show the files involved and show feedback from the IDE reporting successful compilation and linking; if the project is compiled on the command line, the screenshot must show the commands issued; if a makefile was used, include a copy of the makefile in addition to the screenshot.

screenshot of program run on the console

Copy the executable to a location whose path contains a recognizable version of your name. If working with a group, make sure that the path does not contain any version of the names of any groupmates. Create a new folder named after yourself as needed. Run the executable by typing its name at the command prompt, and take a screenshot as the executable starts. Do not run the application from within an IDE.

Code locator

Fill in the table locating the specified code items. Fill in this table after the code has been finalized. Inaccurate details will be penalized.

code items	file name	line numbers
separate processing of first line of signal file		
input validation of integers in processing signal files and system files		
input validation of floating point numbers in processing signal files and system files		
allocation of fixed-size arrays for simulating LTI systems		
simulation function(s) (that compute the output(s) given input(s) to the system)		

Do a self-evaluation using the project evaluation table and evaluation procedures provided. Fill in your scores and add your points correctly.

Do a self-evaluation using the project evaluation table provided.

Place the screenshot of the basic test, the code locator table, and the self-evaluation table together in a single document and export this document to pdf.

Submit a single zip file containing
* C++ implementation files and header files for the project
* required documentation

Do NOT include executables in your submission.

Project Evaluation		
Item	Points	Rubrics
early work	8/8	(all or nothing) 8: at most 10% of the code in the final implementation differs from that in early work submission
implementation and testing environment report	4/4	4 - all instructions followed correctly
Basic Test	50/50	0 - at least one computation in the basic test is not done correctly or C++ vectors or equivalent structures are used in simulating an LTI system 50 - correctly simulates the LTI system provided in the basic test for all inputs provided
	6/6	6 - screenshot submitted showing the correctly computed normalized crosscorrelation as it appears on the console output
generality of simulation	6/6	6 - LTI systems are simulated correctly and efficiently at all times
DFT function	6/6	6 - LTI systems are simulated correctly and efficiently at all times, by function(s) as specified
processing of signal files	4/4	4 - The file is opened only once and the first line of the signal file is processed correctly and separately from code processing the rest of the signal file
processing of system files	4/4	4 - The file is opened only once and the first values of the file are used as bases for allocating a fixed amount of memory for simulating the system
input validation	4/4	4 - integers and floating point numbers are fully validated when signal and system files are processed
quality of application, user interface and program flow	4/4	areas to check (4): robustness of application, correct program flow, appropriate feedback and outputs, proper support of all required commands 0 - application sometimes behaves erratically (not robust), or deficiencies in two other areas 2 - robust application, deficiencies in at most one other area 4 - excellent in all areas
self-evaluation	4/4	4 - self-evaluation accurate (or evaluating this item leads to an error)
total	100/100	

Evaluation Procedures

Special Cases

Score for the entire project is 0 if
* no source files are submitted
* the source files, as submitted, will not compile as a C++ project
* the implementation uses resources that are not allowed

early work

Early work credits are based on the final code submitted, assuming the final and early work code use the same style. The total number of lines in the final code that are not in the early work version, or that were modified from the early work version, other than formatting changes that do not modify compiled code, must not be more than 10% of the number of lines in the final code, excluding blank lines.

basic test

The correct result must have been computed from the provided test data, not hardcoded in any manner. Score 0 if there is any anomaly in the basic test.

screenshot

must show the command line and the complete results produced on the console; values must be the same as expected. Score 0 if the complete results are not all visible, or values are not the same as expected, or screenshot shows the contents of the output file instead of the console output. Score 0 if there is any anomaly in the basic test.

Simulation

0 - LTI systems are sometimes not simulated correctly or C++ vectors or equivalent structures are used in simulating an LTI system

processing of signal files

Score 0 if signal files are opened more than once.

Score 0 if the first line is not processed correctly or not processed separately from the rest of the signal file. Check that a “1” as the first token on the first line is interpreted differently from a “1.” .

Check that data extracted are put into dynamically allocated arrays of doubles. Check that contents of vectors used while extracting data from signal files are copied into arrays. Do more tests if in doubt. Insert comments. Insert blank lines. Test with invalid files. Score 0 if any deficiency manifests. Scale points by half if related code elements are not located accurately.

processing of system files

Score 0 if system files are opened more than once.

Check that integers specifying the size of the system are used correctly to allocate arrays of the correct size for storing coefficients, inputs, and outputs. Insert comments. Insert blank lines. Test with invalid files. Score 0 if any deficiency manifests. Scale points by half if related code elements are not located accurately.

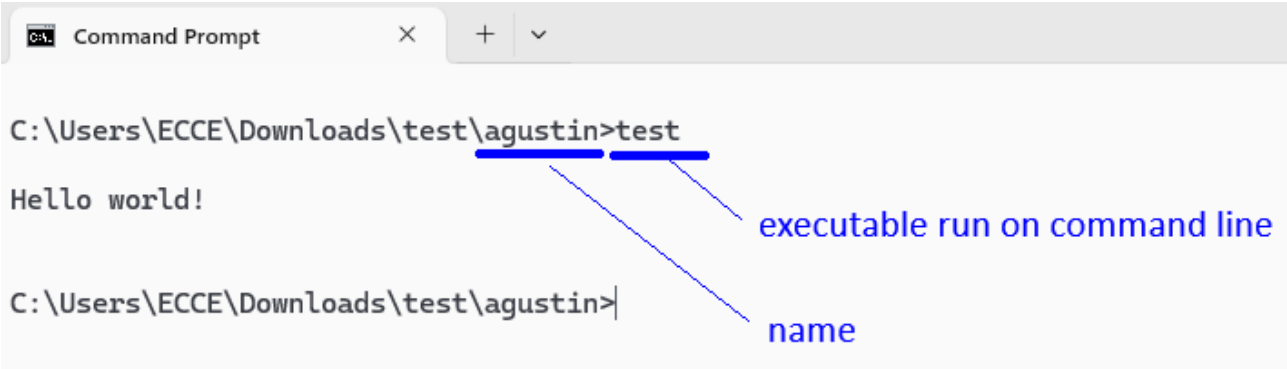
input validation

Score 0 if not done.
Make a signal value invalid by adding an invalid character to a signal value (e.g. -2a). Check that this is treated as a comment and the signal extracted shorter in duration than if there were no invalid character. Do similar tests on system files. Do more tests if in doubt. Score 0 if any deficiency manifests. Scale points by half if related code elements are not located accurately.

command line
Check that application accepts filenames specified on the command line. Change filename extensions and check that the application accepts them (e.g. remove the .txt, change .txt to .exe or .jpg, etc). Score 0 if there are deficiencies.

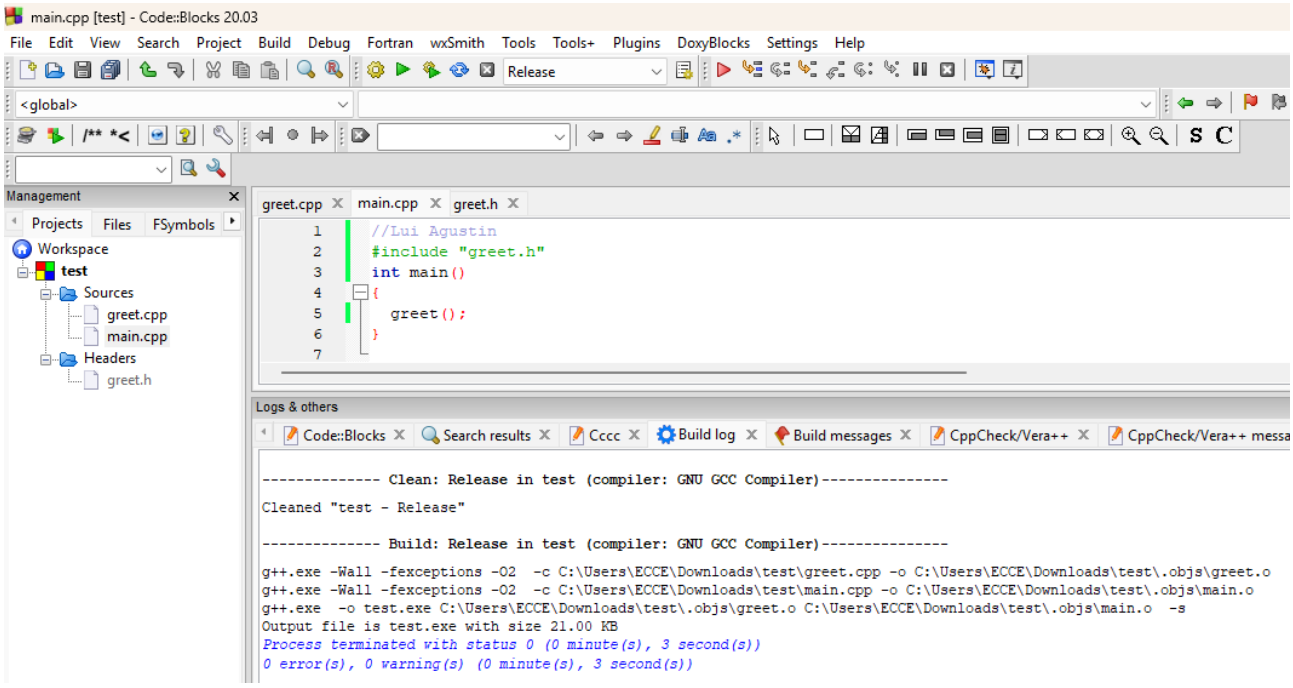
application feedback
Score 0 if application does not produce required output or provide feedback in some instances, or feedback from application misses some details. Score 0 if application ever crashes, goes into an infinite loop, or behaves erratically.

Implementation and Testing Environment (4 pts)
1 - all details of implementation platform are provided
1 - screenshot of compilation: all instructions are followed correctly
1 - screenshot of program run on the console: all instructions are followed correctly
1 - all filenames appearing in the screenshots are consistent with files in the project submission



sample screenshot - executable test.exe run by typing its name at the prompt from a path with name of author

ENGG 151.01 - Signals, Spectra, and Signal Processing, Lecture
ENGG 151.01 A 2025-2
Luisito L. Agustin

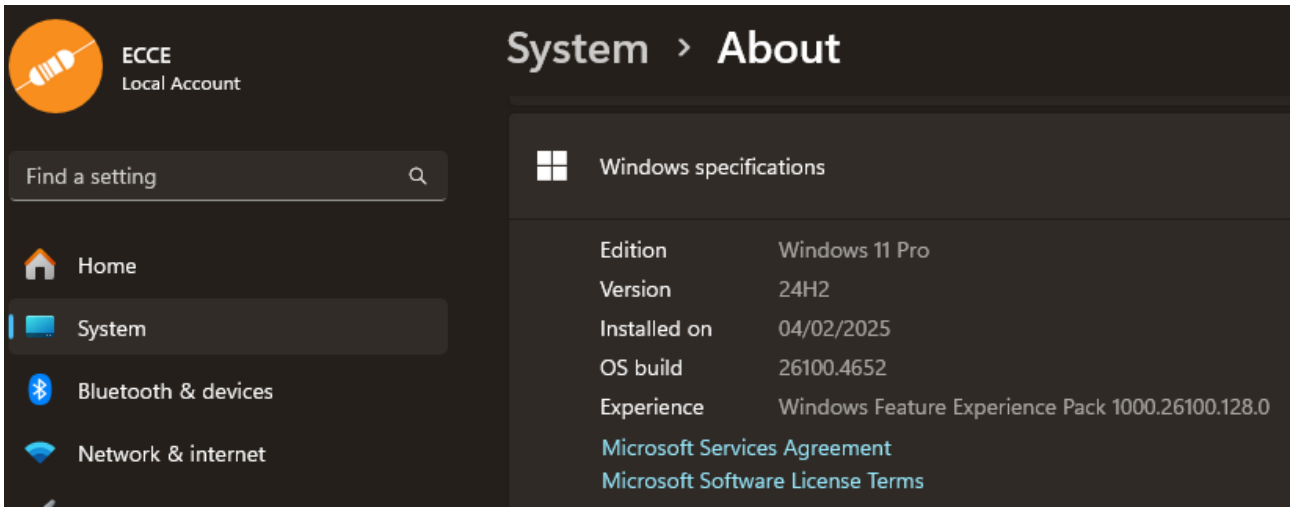


sample screenshot - compilation on Code::Blocks

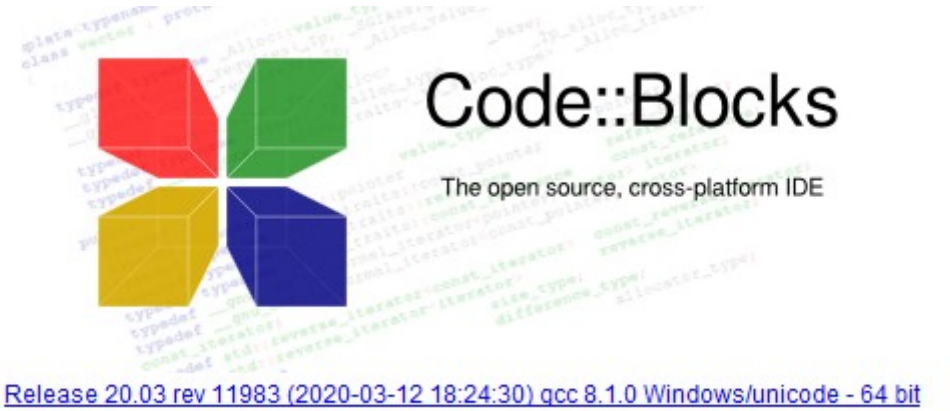
```
C:\Users\ECCE\codeblocks\MinGW\bin>g++ -v
Using built-in specs.
COLLECT_GCC=g++
COLLECT_LTO_WRAPPER=C:/Users/ECCE/codeblocks/MinGW/bin/../libexec/gcc/x86_64-w64-mingw32/8.1.0/lto-wrapper.exe
Target: x86_64-w64-mingw32
Configured with: ../../../src/gcc-8.1.0/configure --host=x86_64-w64-mingw32 --build=x86_64-w64-mingw32 --target=x86_64-w64-mingw32 --prefix=/mingw64
 --with-sysroot=/c/mingw810/x86_64-810-posix-seh-rt_v6-rev0/mingw64 --enable-shared --enable-static --disable-multilib --enable-languages=c,c++,fort
ran,lto --enable-libstdcxx-time=yes --enable-threads=posix --enable-libgomp --enable-libatomic --enable-lto --enable-graphite --enable-checking=rele
ase --enable-fully-dynamic-string --enable-version-specific-runtime-libs --disable-libstdcxx-pch --disable-libstdcxx-debug --enable-bootstrap --disa
ble-rpath --disable-win32-registry --disable-nls --disable-werror --disable-symvers --with-gnu-as --with-gnu-ld --with-arch=nocona --with-tune=core2
 --with-libiconv --with-system-zlib --with-gmp=/c/mingw810/prerequisites/x86_64-w64-mingw32-static --with-mpfr=/c/mingw810/prerequisites/x86_64-w64-
mingw32-static --with-mpc=/c/mingw810/prerequisites/x86_64-w64-mingw32-static --with-isl=/c/mingw810/prerequisites/x86_64-w64-mingw32-static --with-
pkgversion='x86_64-posix-seh-rev0, Built by MinGW-W64 project' --with-bugurl=https://sourceforge.net/projects/mingw-w64 CFLAGS='-O2 -pipe -fno-ident
 -I/c/mingw810/x86_64-810-posix-seh-rt_v6-rev0/mingw64/opt/include -I/c/mingw810/prerequisites/x86_64-zlib-static/include -I/c/mingw810/prerequisite
s/x86_64-w64-mingw32-static/include' CXXFLAGS='-O2 -pipe -fno-ident -I/c/mingw810/x86_64-810-posix-seh-rt_v6-rev0/mingw64/opt/include -I/c/mingw810/
prerequisites/x86_64-zlib-static/include -I/c/mingw810/prerequisites/x86_64-w64-mingw32-static/include' CPPFLAGS=' -I/c/mingw810/x86_64-810-posix-se
h-rt_v6-rev0/mingw64/opt/include -I/c/mingw810/prerequisites/x86_64-zlib-static/include -I/c/mingw810/prerequisites/x86_64-w64-mingw32-static/includ
e' LDFLAGS='-pipe -fno-ident -L/c/mingw810/x86_64-810-posix-seh-rt_v6-rev0/mingw64/opt/lib -L/c/mingw810/prerequisites/x86_64-zlib-static/lib -L/c/m
ingw810/prerequisites/x86_64-w64-mingw32-static/lib '
Thread model: posix
gcc version 8.1.0 (x86_64-posix-seh-rev0, Built by MinGW-W64 project)

C:\Users\ECCE\codeblocks\MinGW\bin>
```

compiler - gcc version 8.1.0 (x86_64-posix-seh-rev0, Built by MinGW-W64 project)



operating system - Windows 11 Pro, 24H2, build 26100.4652



IDE -Code::Blocks 20.03
rev 11983 (2020-03-12 18:24:30) gcc 8.1.0 Windows/unicode - 64 bit

self-evaluation
0 - not done
2 - self-evaluation score differs from project score by more than 10 points or scores not tallied properly
4 - self-evaluation accurate (or evaluating this item leads to an error)

Notes

Possible prototypes for simulator with multiple input signal values:

```
compute_outputs ( double * acoef, double * bcoeff,
  double * inputs, double * outputs, int sizea, int sizeb,
  double * input_samples, int nSamples,
  double ** output_samples);

ltiSystem::compute_outputs ( double * input_samples,
  int nSamples, double ** output_samples);
// The LTI System arrays and their sizes would be
// implied parameters accessible to all member functions
// of the ltiSystem class.
```

These may be implemented by repeatedly invoking the required function taking a single input sample, but are better implemented without the overhead of repeated function calls.

Fixed Size Arrays

The first two lines of LTI System Files provide $M+1$, the number of nonrecursive coefficients, and N, the number of recursive coefficients. The number of nonrecursive coefficients would also be the number of present and past inputs that need to be retained in memory. The number of recursive coefficients is also the number of outputs that need to be retained in memory.

Once valid values of $M+1$ and N have been obtained from an LTI System File, the application should allocated the required arrays of sizes $M+1$ and N using the `new` operator and have pointers (`double *`) point to them.

The LTI System File parser must then attempt to extract the correct number of coefficients and store them in the arrays. If the correct number of coefficients is obtained, the previous coefficients are discarded (use `delete[]` since they were presumably allocated using `new`), and pointers updated to point to the new coefficient arrays.

Initial Conditions

Specification of initial conditions may be supported by an additional command:

initcond *filename* (The word "initcond" typed on the prompt provided by the application; The rest of the line must contain a valid filename.)	Try to extract initial conditions from *filename*. Possible file format: A text file where the first $M+N$ lines contain the initial conditions in the following order: x(0) optional comments x(1) optional comments . . . x(M-1) optional comments y(-1) optional comments y(-2) optional comments . . . y(-N) optional comments optional comments The file is not valid if the correct number of initializers cannot be extracted from the file. If the file is valid, log the line "clear". Simulation restarts. Display an error message if no LTI system has been specified yet (e.g. at start of application) Note that x(0) becomes x(-1) when the actual x(0) is specified, x(-1) becomes x(-2), and so on.

Appendices

(see next pages)

Signal File Format

LTI System File Format

Signal File Format

Luisito L. Agustin
2023 01 18

Introduction

This document specifies a text file format for representing finite-duration signals. In this file format, signal values are expected to be floating point values in general.

All signal values of a finite-duration signal, $x(n)$, are zero except for values of the integer index n in a finite range from $n = start$ to $n = end$ where $start < end$. The signal is then said to have a *duration* given by

$$duration = end - start + 1 \ .$$

A signal file is a text file regardless of whatever extension may be used in the filename.

Format

The format for a signal file is as follows:

[optional starting index] **signal value** optional comments **signal value** optional comments . . . **signal value** optional comments optional comments

The finite set of signal values must be the first or leftmost tokens in consecutive lines of the text file starting with the first line of the file, except that the first signal value (on the first line) may be preceded by an integer indicating the index $n = start$ of the signal value that follows on the line. Each signal value may be preceded by any amount of white space on each line. Only the starting index needs to be specified. It is understood that succeeding signal values correspond to successive increments of the index.

Any text that follows a signal value is considered a comment and may be ignored. A line which does not start with a valid floating point value is a comment. A signal file does not encode any valid signal if the first line does not start with a valid integer or floating point number.

The duration of the signal is determined from the number of consecutive lines from which a valid signal value can be extracted. The signal values either terminate when the end of a file is reached or when the next line extracted from the file does not contain a valid signal value as its first token.

If the optional starting index is absent, the implied starting index is 0. That is, if the first item on the first line is not a valid integer but is a valid floating point number, then the start index of the signal is zero and the signal value at $n = 0$ is the floating point number extracted from the first line.

If the first line contains a single numerical value, that numerical value must be a signal value and the implied start index is 0.

Examples

```
-4 2 This is a comment to be ignored
-1 Comment: the -4 on the first line means the signal starts with x(-4) = 2
3
7
1
2
-3
This line does not start with a floating point value.
The parser will conclude that -3 is the end of the finite duration signal.
Everything that comes after may be ignored.
This signal is the sequence x(n) from Example 2.6.1 in
John Proakis and Dimitris Manolakis: Digital Signal Processing, 4th ed, 2007.
x(-4) = 2
x(-3) = -1
x(-2) = 3
x(-1) = 7
x(0) = 1
x(1) = 2
x(2) =-3

start = -4
end = 2
duration = 7  ( = 2 - (-4) + 1 )
```

Without comments, the signal file would simply contain:

```
-4 2
-1
3
7
1
2
-3
```

Comments do not change the signal being represented.

Notes

Some confusion may be possible in the first line if there is no starting index (implied start = 0) but the floating point signal value appears as an integer. The signal value might be incorrectly interpreted as the start index.

Recommendations for Handling Signal Files

The extraction operator (>>) in C++ skips white space and terminates processing on any white space character. It would skip whatever leading white space there might be before the signal values.

The first line of a signal file needs to be processed differently from the rest of a signal file. It should not be processed as part of a loop.

There's no limit to how large a signal file could be. Applications should not impose any limit. Nothing in the file format specifies the duration of the signal. A C++ vector should be used to store the signal values obtained until the last signal value is extracted from the file, at which point the duration would be known. Once the duration is known, an array must be dynamically allocated and the contents of the vector copied to the dynamically allocated array. Any processing to be done must be done on the dynamically allocated array. Processing arrays is faster than processing vectors.

Whether a signal file is valid depends on its contents, never on the extension used for the filename. While a .txt extension may be common for text files, applications that process signal files should neither expect nor require a .txt extension. They should work with whatever filename is provided, as is, without any modification.

A signal file should be opened only once, and its lines processed one at a time from start to finish without any need to process any line twice.

Suggested C++ Signal Class API

```
class engg151Signal
{
  public:
    engg151Signal ();
    // default constructor
    // suggested default: one-element signal with value 0.0, start index 0

    engg151Signal ( double * x, int start, int duration );

    bool importSignalFromFile ( string filename);
    // returns true if a valid signal was actuallly obtained from filename
    // returns false otherwise

    bool exportSignalToFile ( string filename );
    // returns true if the signal was successfully exported to a file
    // returns false otherwise

    int start();
    int end();
    int duration();
    // as implied by the names,
    // return the start index, end index, and duration of the signal
    // respectively

    double * data();
    // returns a pointer to an array of double containing the signal values
}

engg151Signal normalizedXCorr ( engg151Signal x, engg151Signal y);
// computes the normalized crosscorrelation of x crosscorrelated with y
// and returns the normalized crosscorrelation as a signal object.
// Note that the crosscorrelation is not commutative.
// This does not need to be a class member.
```

Revision History - Signal File Format

2021 02 17	first version
2023 01 18	revisions for clarity, recommendations

LTI System File Format

Luisito L. Agustin
2022 04 13

Introduction

A general form of the input-output relationship for a linear time-invariant (LTI) system is given by the linear constant-coefficient difference equation

$$y(n) = -\sum_{k=1}^{N} a_k\, y(n-k) + \sum_{k=0}^{M} b_k\, x(n-k) \ ,$$

where the $\{a_k\}$ and $\{b_k\}$ are constant parameters that specify the system and are independent of the input $x(n)$ and output $y(n)$. For a fixed value n, the equation specifies how the output $y(n)$ is computed based on the parameters $\{ a_k \}$ and $\{ b_k \}$, current and past inputs $\{ x(n-k) \}$ and past outputs $\{ y(n-k) \}$.

File Format

The parameters of an LTI system shall be read from a line-oriented text file with the following format:

```
==========================================================================
M+1    optional comments This is the number of nonrecursive coefficients
N      optional comments This is the number of recursive coefficients
b_0    optional comments
b_1    optional comments
.
.
.
b_M    optional comments
a_1    optional comments
.
.
.
a_N    optional comments
optional comments...
==========================================================================
```

The horizontal bars (===...====) are not part of the format.

Each required item in the file starts a new line. There may be any amount of white space before the actual required item. The rest of the line is treated as a comment and may be ignored.

M and *N* are used the same way in the file format specification as in the equation shown. *M+1* is the number of nonrecursive coefficients. *N* is the number of recursive coefficients. *M+1* and *N* must both be nonnegative integers. *M+1* and *N* cannot both be zero.

These two integers are followed by *M+N+1* floating point coefficients interpreted as shown in the file format. Anything that follows the required number of coefficients may be ignored.

Notes

sample LTI system file

```
3                       nNonRecursiveCoefs = M+1 = 3; M = 2
2                       nRecursiveCoefs = N = 2
0.3290316064918401 b0 = 0.3290316064918401
-0.31460820551056307        b1 = -0.31460820551056307
0.11124319871419965         b2 = 0.11124319871419965
-1.439788397628722          a1 = -1.439788397628722
0.5654549973241987 a2 = 0.5654549973241987
from DSP Lab Activity: Magnitude Response of a Biquad Filter
```

without comments

```
3
2
0.3290316064918401
-0.31460820551056307
0.11124319871419965
-1.439788397628722
0.5654549973241987
```

C++ Notes

A *white space* is any character `c`, for which `isspace( c)` returns true.

The extraction operator (>>) in C++ will skip white spaces before attempting to extract information of the required type. The operator will also terminate on a white space.

References

[Proakis] (book) John Proakis and Dimitris Manolakis: Digital Signal Processing, 4th ed, 2007. *(Chapter 2 discusses linear time-invariant systems. Equation 2.3.1 gives the general form of the input-output relationship.)*

Revision History - LTI System File Format

2022 01 27	formalized 1st version, format was in use since 2020-2021
2022 04 13	added introduction for context, minor revisions for clarity